

ArtConnect — Livrable Étape 3

Implémentation BD & Fonctionnalités Avancées

Bases de données 2 — TI603 — INGE1 — EFREI Paris 2025/2026

Rouvrais Clément/ Maxime Brun/ Safiane Ras-el-Qdim/ Luc Martrenchar



1. Prompt utilisé pour la génération des données

Les données d'exemple ont été générées avec l'aide d'un LLM (Claude Sonnet). Le prompt suivant a été soumis, puis le script produit a été vérifié et adapté manuellement.

1.1 Prompt soumis au LLM

```
Tu es un expert MySQL. Génère un script SQL d'insertion de données réalistes pour la base de données ArtConnect dont le schéma contient les tables suivantes : artist, artwork, discipline, artist_discipline, gallery, exhibition, exhibition_artwork, workshop, community_member, member_discipline, booking, review.
```

Contraintes :

- 10 artistes avec des nationalités et disciplines variées (peinture, sculpture, photographie, art numérique, céramique, musique)
- 15 oeuvres réparties entre les artistes, avec des statuts variés (FOR_SALE, SOLD, EXHIBITED)
- 4 galeries dans des villes européennes différentes
- 5 expositions dont 3 en cours et 1 à venir
- 6 ateliers à venir, animés par des artistes différents
- 8 membres de la communauté avec des adhésions free et premium
- Des réservations croisées (plusieurs membres par atelier)
- 10 avis sur des oeuvres, avec des notes entre 3 et 5

Format attendu : uniquement des instructions INSERT INTO avec ON DUPLICATE KEY UPDATE. Utilise des valeurs cohérentes avec les contraintes de clés étrangères. Réponds en SQL uniquement, sans commentaires ni explications.

1.2 Ajustements manuels apportés

- Remplacement des INSERT directs par des sous-requêtes SELECT pour respecter les FK (les id AUTO_INCREMENT ne sont pas connus à l'avance)
- Ajout des clauses ON DUPLICATE KEY UPDATE sur toutes les tables pour rendre le script idempotent
- Correction de l'enum status dans artwork (FOR_SALE / SOLD / EXHIBITED)
- Vérification que toutes les dates d'ateliers sont dans le futur (NOW() + INTERVAL)

- Ajout des tags d'oeuvres et des disciplines favorites des membres, absents du script LLM

2. Script d'insertion des données d'exemple

Le script ci-dessous insère 10 artistes, 15 oeuvres, 4 galeries, 5 expositions, 6 ateliers, 8 membres avec leurs réservations et avis. Il est idempotent (ON DUPLICATE KEY UPDATE).

Exécution : `mysql -u root -p artconnect_db < data_seed.sql`

Le script d'insertion d'exemple est présent dans le dossier sur github.

3. Documentation des objets BD avancés

3.1 Vues

Trois vues ont été créées, chacune avec un objectif fonctionnel précis.

Vue	Objectif	Tables sources
v_artist_portfolio	Résumé public : masque email et téléphone (sécurité). Agrège disciplines et valeur totale du portfolio.	artist, artist_discipline, discipline, artwork
v_exhibition_programme	Programme des expositions en cours avec jours restants.	exhibition, gallery, exhibition_artwork
v_workshop_availability	Ateliers à venir avec places restantes et statut (DISPONIBLE / PRESQUE COMPLET / COMPLET).	workshop, artist, booking

v_artist_portfolio — extrait

```
CREATE VIEW v_artist_portfolio AS
SELECT a.id, a.name, a.bio, a.city, a.website,
       GROUP_CONCAT(DISTINCT d.name ORDER BY d.name SEPARATOR ', ') AS disciplines,
       COUNT(DISTINCT aw.id) AS nb_artworks,
       COALESCE(SUM(aw.price),0) AS total_portfolio_value
FROM artist a
LEFT JOIN artist_discipline ad ON a.id = ad.artist_id
LEFT JOIN discipline d        ON d.id  = ad.discipline_id
LEFT JOIN artwork aw          ON aw.artist_id = a.id
WHERE a.is_active = TRUE
GROUP BY a.id, a.name, a.bio, a.city, a.website;
```

v_workshop_availability — extrait

```
CREATE VIEW v_workshop_availability AS
SELECT w.id, w.title, w.date_time, w.price, w.level, a.name AS instructeur,
       w.max_participants,
       COUNT(b.id) AS reservations_confirmees,
       w.max_participants - COUNT(b.id) AS places_restantes,
       CASE
         WHEN w.max_participants - COUNT(b.id) = 0 THEN 'COMPLET'
         WHEN w.max_participants - COUNT(b.id) <= 2 THEN 'PRESQUE COMPLET'
         ELSE 'DISPONIBLE'
       END AS disponibilite
FROM workshop w
```

```
JOIN artist a ON a.id = w.instructor_id
LEFT JOIN booking b ON b.workshop_id = w.id AND b.payment_status != 'CANCELLED'
WHERE w.date_time > NOW()
GROUP BY w.id, w.title, w.date_time, w.price, w.level, a.name, w.max_participants;
```

3.2 Index

Sept index ont été créés pour optimiser les requêtes les plus fréquentes.

Index	Table / Colonne	Justification
idx_artist_city	artist(city)	Filtrage fréquent par ville dans l'interface de recherche
idx_artwork_artist	artwork(artist_id)	Jointure artwork ↔ artist, présente dans presque toutes les requêtes
idx_artwork_status	artwork(status)	Filtre sur les oeuvres à vendre ou exposées
idx_exhibition_end	exhibition(end_date)	Sélection des expositions en cours (end_date >= CURDATE)
idx_booking_member	booking(member_id)	Historique des réservations d'un membre
idx_booking_workshop	booking(workshop_id)	Comptage des participants à un atelier
idx_review_artwork	review(artwork_id)	Calcul de note moyenne par oeuvre

3.3 Triggers

Quatre triggers assurent l'intégrité des données et la traçabilité.

Trigger	Événement	Rôle
trg_check_booking_capacity	BEFORE INSERT ON booking	Bloque toute réservation si l'atelier est complet. Lève SQLSTATE 45000.
trg_check_exhibition_dates_insert	BEFORE INSERT ON exhibition	Vérifie que end_date >= start_date à la création.
trg_check_exhibition_dates_update	BEFORE UPDATE ON exhibition	Même vérification à la modification.
trg_audit_exhibition_update	AFTER UPDATE ON exhibition	Journalise dans audit_exhibition toute modification de titre ou de date de fin.

trg_check_booking_capacity — extrait

```
DELIMITER //
CREATE TRIGGER trg_check_booking_capacity
BEFORE INSERT ON booking FOR EACH ROW
BEGIN
    DECLARE current_count INT;
    DECLARE max_cap INT;
    SELECT COUNT(*) INTO current_count FROM booking
        WHERE workshop_id = NEW.workshop_id AND payment_status != 'CANCELLED';
    SELECT max_participants INTO max_cap FROM workshop WHERE id = NEW.workshop_id;
    IF current_count >= max_cap THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Atelier complet : capacité maximale atteinte.';
    END IF;
END //
DELIMITER ;
```

trg_audit_exhibition_update — extrait

```
DELIMITER //
CREATE TRIGGER trg_audit_exhibition_update
AFTER UPDATE ON exhibition FOR EACH ROW
BEGIN
    IF OLD.title != NEW.title OR OLD.end_date != NEW.end_date THEN
        INSERT INTO audit_exhibition
            (exhibition_id, old_title, new_title, old_end_date, new_end_date)
        VALUES (OLD.id, OLD.title, NEW.title, OLD.end_date, NEW.end_date);
    END IF;
END //
DELIMITER ;
```

3.4 Fonctions et procédures stockées

Deux fonctions scalaires et trois procédures encapsulent les opérations métier principales.

Objet	Type	Paramètres	Rôle
fn_workshop_participants	FUNCTION	IN: workshop_id	Retourne le nombre de participants confirmés (PAID).
fn_avg_rating_artwork	FUNCTION	IN: artwork_id	Retourne la note moyenne d'une oeuvre (NULL si aucun avis).
sp_create_workshop	PROCEDURE	IN: titre, date, durée, max, prix, artiste, lieu, niveau — OUT: new_id	Crée un atelier après vérification que l'artiste est actif.
sp_register_member	PROCEDURE	IN: email_membre, titre_atelier — OUT: booking_id	Inscrit un membre à un atelier via email et titre.
sp_cancel_booking	PROCEDURE	IN: booking_id — OUT: result	Annule une réservation (statut → CANCELLED) sans suppression.

sp_create_workshop — extrait

```
DELIMITER //
CREATE PROCEDURE sp_create_workshop(
  IN p_title VARCHAR(300), IN p_date_time DATETIME,
  IN p_duration INT, IN p_max_participants INT,
  IN p_price DECIMAL(10,2), IN p_instructor_name VARCHAR(200),
  IN p_location VARCHAR(300), IN p_level VARCHAR(50),
  OUT p_new_id BIGINT)
BEGIN
  DECLARE v_instructor_id BIGINT;
  SELECT id INTO v_instructor_id FROM artist
    WHERE name = p_instructor_name AND is_active = TRUE;
  IF v_instructor_id IS NULL THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Artiste instructeur introuvable ou inactif.';
  END IF;
  INSERT INTO workshop (title, date_time, duration_minutes, max_participants,
    price, instructor_id, location, level)
  VALUES (p_title, p_date_time, p_duration, p_max_participants,
    p_price, v_instructor_id, p_location, p_level);
  SET p_new_id = LAST_INSERT_ID();
END //
DELIMITER ;
```

4. Scripts de test des transactions

Six scénarios transactionnels testent l'atomicité, l'isolation et la robustesse de la base. Chaque scénario est délimité par START TRANSACTION / COMMIT ou ROLLBACK.

Scénario	Description	Résultat attendu
1 — Multi-booking	Inscription d'un membre à 2 ateliers simultanément.	COMMIT : 2 bookings créés ou aucun.
2 — Nouvelle exposition	Création d'une exposition + liaison avec 4 oeuvres en une transaction.	COMMIT : exposition + 4 lignes exhibition_artwork.
3 — Annulation + remboursement	Statut → CANCELLED + enregistrement dans refund_log atomiquement.	COMMIT : statut et log cohérents.
4 — Test ROLLBACK trigger	Tentative de surréservation — le trigger BEFORE INSERT doit bloquer.	ROLLBACK déclenché par SQLSTATE 45000.
5 — Mise à jour prix	Mise à jour du prix d'un atelier avec validation (prix > 0).	COMMIT si valide, ROLLBACK sinon.
6 — Procédures chaînées	sp_create_workshop + 2× sp_register_member dans une seule transaction.	COMMIT : atelier + 2 bookings créés ensemble.

```
-- =====
-- ArtConnectPro – Scripts de test des transactions
-- =====
-- Chaque scénario est atomique : toutes les opérations
-- réussissent ou aucune n'est validée (ROLLBACK).
-- =====

USE artconnect_db;

-- =====
-- SCÉNARIO 1 : Inscription d'un membre à plusieurs ateliers
-- Atomicité : soit tous les bookings sont créés, soit aucun.
-- =====

START TRANSACTION;

SET @member_id = (SELECT id FROM community_member WHERE email =
'alice.dupont@gmail.com');
SET @ws1_id     = (SELECT id FROM workshop WHERE title = 'Dessin hyperréaliste au
crayon');
SET @ws2_id     = (SELECT id FROM workshop WHERE title = 'Sculpture sur bronze –
Techniques avancées');

-- Vérification préalable : le membre existe
IF @member_id IS NULL THEN
    ROLLBACK;
END IF;

INSERT INTO booking (workshop_id, member_id, booking_date, payment_status)
VALUES (@ws1_id, @member_id, NOW(), 'PAID');

INSERT INTO booking (workshop_id, member_id, booking_date, payment_status)
VALUES (@ws2_id, @member_id, NOW(), 'PAID');

-- Vérification : les deux lignes sont bien créées
SELECT 'Scénario 1 – bookings créés :' AS message, COUNT(*) AS nb
```

```

FROM booking
WHERE member_id = @member_id
    AND workshop_id IN (@ws1_id, @ws2_id)
    AND payment_status = 'PAID';

COMMIT;
SELECT 'Scénario 1 COMMIT OK' AS resultat;

-- =====
-- SCÉNARIO 2 : Ajout d'une nouvelle exposition avec ses œuvres
-- Atomicité : la galerie, l'exposition et les liaisons
--             exhibition_artwork sont créées ensemble.
-- =====

START TRANSACTION;

-- Insertion de l'exposition
INSERT INTO exhibition (title, start_date, end_date, gallery_id, curator_name, theme,
description)
SELECT 'Futur Antérieur',
    DATE_ADD(CURDATE(), INTERVAL 30 DAY),
    DATE_ADD(CURDATE(), INTERVAL 120 DAY),
    id, 'Nadia Kowalski', 'Technologie et humanité',
    'Dialogue entre créations humaines et intelligences artificielles.'
FROM gallery WHERE name = 'The Frame Gallery';

SET @new_exh_id = LAST_INSERT_ID();

-- Association avec des œuvres existantes
INSERT INTO exhibition_artwork (exhibition_id, artwork_id)
SELECT @new_exh_id, id FROM artwork
WHERE title IN ('Algorithmes #7', 'Codex Aztèque 2077', 'Glitch Tenochtitlán', 'Raga
Électrique');

-- Contrôle
SELECT 'Scénario 2 – exposition créée :' AS message, title, start_date FROM exhibition
WHERE id = @new_exh_id;
SELECT 'Scénario 2 – œuvres liées :' AS message, COUNT(*) AS nb FROM exhibition_artwork
WHERE exhibition_id = @new_exh_id;

COMMIT;
SELECT 'Scénario 2 COMMIT OK' AS resultat;

-- =====
-- SCÉNARIO 3 : Annulation d'une réservation avec remboursement
-- Atomicité : le statut passe à CANCELLED et on enregistre
--             l'événement dans une table de remboursements.
-- =====

-- Table de remboursements (créée si absente)
CREATE TABLE IF NOT EXISTS refund_log (
    id BIGINT NOT NULL AUTO_INCREMENT,
    booking_id BIGINT NOT NULL,
    amount DECIMAL(10,2),
    refunded_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    reason VARCHAR(300),
    PRIMARY KEY (id)
);

START TRANSACTION;

SET @cancel_booking_id = (
    SELECT b.id FROM booking b
    JOIN workshop w ON w.id = b.workshop_id
    JOIN community_member m ON m.id = b.member_id
    WHERE m.email = 'bob.martin@outlook.com'
    AND w.title = 'Sculpture sur bronze – Techniques avancées'

```

```

        AND b.payment_status = 'PAID'
    LIMIT 1
);

-- Annulation
UPDATE booking
SET payment_status = 'CANCELLED'
WHERE id = @cancel_booking_id;

-- Enregistrement du remboursement
INSERT INTO refund_log (booking_id, amount, reason)
SELECT @cancel_booking_id, w.price, 'Annulation membre - demande volontaire'
FROM booking b JOIN workshop w ON w.id = b.workshop_id
WHERE b.id = @cancel_booking_id;

-- Contrôle
SELECT 'Scénario 3 - statut après annulation : ' AS message, payment_status
FROM booking WHERE id = @cancel_booking_id;
SELECT 'Scénario 3 - remboursement enregistré : ' AS message, amount, reason
FROM refund_log WHERE booking_id = @cancel_booking_id;

COMMIT;
SELECT 'Scénario 3 COMMIT OK' AS resultat;

-- =====
-- SCÉNARIO 4 : Test de ROLLBACK - tentative de surréservation
-- Ce scénario DOIT échouer grâce au trigger trg_check_booking_capacity.
-- Le ROLLBACK garantit qu'aucun état partiel n'est conservé.
-- =====

START TRANSACTION;

-- On récupère un atelier avec peu de places (max_participants = 5)
SET @ws_bronze = (SELECT id FROM workshop WHERE title = 'Sculpture sur bronze -
Techniques avancées');

-- Tentative d'insérer 10 réservations alors que max = 5
-- Le trigger lèvera une erreur dès que la capacité sera atteinte.

-- Bloc conditionnel de test (simulé en SQL pur)
SELECT CONCAT(
    'Places actuellement prises : ',
    fn_workshop_participants(@ws_bronze),
    ' / ',
    (SELECT max_participants FROM workshop WHERE id = @ws_bronze)
) AS etat_avant;

-- Tentative d'insertion qui doit déclencher le trigger
BEGIN
    DECLARE CONTINUE HANDLER FOR SQLSTATE '45000'
    BEGIN
        SELECT 'Scénario 4 - TRIGGER ACTIVÉ : surréservation bloquée.' AS resultat;
        ROLLBACK;
    END;

    INSERT INTO booking (workshop_id, member_id, booking_date, payment_status)
    VALUES (@ws_bronze, (SELECT id FROM community_member WHERE email = 'mei@chen.cn'
LIMIT 1), NOW(), 'PENDING');
    -- Si on arrive ici sans erreur, on commit normalement
    COMMIT;
    SELECT 'Scénario 4 - place disponible, booking enregistré.' AS resultat;
END;

-- =====
-- SCÉNARIO 5 : Mise à jour atomique du prix d'un atelier
-- avec validation métier (prix ne peut pas être 0)
-- =====

```

```

START TRANSACTION;

SET @ws_nft = (SELECT id FROM workshop WHERE title = 'Créer son premier NFT');
SET @new_price = 85.00;

-- Validation métier
IF @new_price <= 0 THEN
    ROLLBACK;
    SELECT 'Scénario 5 – ROLLBACK : prix invalide (doit être > 0).' AS resultat;
ELSE
    UPDATE workshop SET price = @new_price WHERE id = @ws_nft;

    SELECT 'Scénario 5 – nouveau prix :' AS message, price
    FROM workshop WHERE id = @ws_nft;

    COMMIT;
    SELECT 'Scénario 5 COMMIT OK' AS resultat;
END IF;

-- =====
-- SCÉNARIO 6 : Utilisation des procédures stockées en transaction
-- Enchaînement sp_create_workshop + sp_register_member
-- =====

START TRANSACTION;

CALL sp_create_workshop(
    'Aquarelle pour débutants',
    DATE_ADD(NOW(), INTERVAL 45 DAY),
    150, 12, 65.00,
    'Elena Marchetti',
    'Galerie du Marais – Studio A',
    'Débutant',
    @new_ws_id
);

CALL sp_register_member('david.leroy@gmail.com', 'Aquarelle pour débutants',
    @new_booking_id);
CALL sp_register_member('giulia.romano@hotmail.com', 'Aquarelle pour débutants',
    @new_booking_id2);

SELECT 'Scénario 6 – atelier créé id=' AS message, @new_ws_id AS workshop_id;
SELECT 'Scénario 6 – bookings en attente :' AS message, COUNT(*) AS nb
FROM booking WHERE workshop_id = @new_ws_id;

COMMIT;
SELECT 'Scénario 6 COMMIT OK' AS resultat;

-- =====
-- VÉRIFICATIONS FINALES – Utilisation des vues
-- =====

SELECT '=== v_artist_portfolio (top 3 par valeur) ===' AS section;
SELECT name, city, disciplines, nb_artworks, total_portfolio_value
FROM v_artist_portfolio
ORDER BY total_portfolio_value DESC LIMIT 3;

SELECT '=== v_exhibition_programme (expositions en cours) ===' AS section;
SELECT exposition, galerie, jours_restants, nb_oeuvres
FROM v_exhibition_programme
ORDER BY jours_restants ASC;

SELECT '=== v_workshop_availability ===' AS section;
SELECT title, instructeur, places_restantes, disponibilite
FROM v_workshop_availability

```

```
ORDER BY date_time ASC;
```

```
SELECT '=== Fonction fn_avg_rating_artwork ===' AS section;  
SELECT a.title, fn_avg_rating_artwork(a.id) AS note_moyenne  
FROM artwork a  
WHERE EXISTS (SELECT 1 FROM review r WHERE r.artwork_id = a.id)  
ORDER BY note_moyenne DESC;
```

5. Script complet — Objets BD avancés

Script à exécuter après schema.sql et data_seed.sql.

Exécution : `mysql -u root -p artconnect_db < advanced_objects.sql`

```
-- =====
-- ArtConnectPro - Objets BD avancés
-- Vues, Index, Triggers, Procédures stockées, Fonctions
-- À exécuter APRÈS schema.sql et data_seed.sql
-- =====

USE artconnect_db;

-- =====
-- SECTION 1 - VUES
-- =====

-- -----
-- Vue 1 : v_artist_portfolio
-- Objectif : résumé public d'un artiste avec le nombre d'œuvres
--            et les disciplines, sans données de contact sensibles.
--            Sécurité : masque phone et contact_email.
-- -----
DROP VIEW IF EXISTS v_artist_portfolio;
CREATE VIEW v_artist_portfolio AS
SELECT
    a.id,
    a.name,
    a.bio,
    a.birth_year,
    a.city,
    a.website,
    a.is_active,
    GROUP_CONCAT(DISTINCT d.name ORDER BY d.name SEPARATOR ', ') AS disciplines,
    COUNT(DISTINCT aw.id) AS nb_artworks,
    COALESCE(SUM(aw.price), 0) AS
total_portfolio_value
FROM artist a
LEFT JOIN artist_discipline ad ON a.id = ad.artist_id
LEFT JOIN discipline d        ON d.id = ad.discipline_id
LEFT JOIN artwork aw         ON aw.artist_id = a.id
WHERE a.is_active = TRUE
GROUP BY a.id, a.name, a.bio, a.birth_year, a.city, a.website, a.is_active;

-- -----
-- Vue 2 : v_exhibition_programme
-- Objectif : programme complet des expositions en cours
--            avec galerie, dates et liste des œuvres.
-- -----
DROP VIEW IF EXISTS v_exhibition_programme;
CREATE VIEW v_exhibition_programme AS
SELECT
    e.id AS exhibition_id,
    e.title AS exposition,
    e.start_date,
    e.end_date,
    DATEDIFF(e.end_date, CURDATE()) AS jours_restants,
    g.name AS galerie,
    g.city AS ville_galerie,
    e.curator_name,
    e.theme,
    COUNT(DISTINCT ea.artwork_id) AS nb_oeuvres
FROM exhibition e
JOIN gallery g   ON g.id = e.gallery_id
LEFT JOIN exhibition_artwork ea ON ea.exhibition_id = e.id
```

```

WHERE e.end_date >= CURDATE()
GROUP BY e.id, e.title, e.start_date, e.end_date, g.name, g.city, e.curator_name,
e.theme;

```

```

-- -----
-- Vue 3 : v_workshop_availability
-- Objectif : ateliers à venir avec le nombre de places restantes.
--           Simplifie les requêtes pour l'affichage UI.
-- -----

```

```

DROP VIEW IF EXISTS v_workshop_availability;
CREATE VIEW v_workshop_availability AS
SELECT
    w.id,
    w.title,
    w.date_time,
    w.duration_minutes,
    w.price,
    w.level,
    w.location,
    a.name                                AS instructeur,
    w.max_participants,
    COUNT(b.id)                          AS reservations_confirmees,
    w.max_participants - COUNT(b.id)     AS places_restantes,
    CASE
        WHEN w.max_participants - COUNT(b.id) = 0 THEN 'COMPLET'
        WHEN w.max_participants - COUNT(b.id) <= 2 THEN 'PRESQUE COMPLET'
        ELSE 'DISPONIBLE'
    END AS disponibilite
FROM workshop w
JOIN artist a      ON a.id = w.instructor_id
LEFT JOIN booking b ON b.workshop_id = w.id AND b.payment_status != 'CANCELLED'
WHERE w.date_time > NOW()
GROUP BY w.id, w.title, w.date_time, w.duration_minutes, w.price,
         w.level, w.location, a.name, w.max_participants;

```

```

-- =====
-- SECTION 2 - INDEX
-- =====

```

```

-- Index 1 : recherche d'artistes par ville (filtrage fréquent)
CREATE INDEX idx_artist_city      ON artist(city);

-- Index 2 : recherche d'œuvres par artiste (jointure très fréquente)
CREATE INDEX idx_artwork_artist   ON artwork(artist_id);

-- Index 3 : recherche d'œuvres par statut (FOR_SALE, etc.)
CREATE INDEX idx_artwork_status   ON artwork(status);

-- Index 4 : expositions par date de fin (affichage "en cours")
CREATE INDEX idx_exhibition_end   ON exhibition(end_date);

-- Index 5 : réservations par membre
CREATE INDEX idx_booking_member   ON booking(member_id);

-- Index 6 : réservations par atelier
CREATE INDEX idx_booking_workshop ON booking(workshop_id);

-- Index 7 : avis par œuvre (calcul de note moyenne)
CREATE INDEX idx_review_artwork   ON review(artwork_id);

```

```

-- =====
-- SECTION 3 - TRIGGERS
-- =====

```

```

-- -----
-- Trigger 1 : trg_check_booking_capacity
-- Objectif : avant toute réservation, vérifier que l'atelier
--             n'est pas complet.
-- -----
DROP TRIGGER IF EXISTS trg_check_booking_capacity;
DELIMITER //
CREATE TRIGGER trg_check_booking_capacity
BEFORE INSERT ON booking
FOR EACH ROW
BEGIN
    DECLARE current_count INT;
    DECLARE max_cap        INT;

    SELECT COUNT(*)          INTO current_count
    FROM booking
    WHERE workshop_id = NEW.workshop_id
      AND payment_status != 'CANCELLED';

    SELECT max_participants INTO max_cap
    FROM workshop
    WHERE id = NEW.workshop_id;

    IF current_count >= max_cap THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Atelier complet : capacité maximale atteinte.';
    END IF;
END //
DELIMITER ;

-- -----
-- Trigger 2 : trg_check_exhibition_dates
-- Objectif : s'assurer que la date de fin d'une exposition
--             est postérieure ou égale à sa date de début,
--             à l'insertion et à la modification.
-- -----
DROP TRIGGER IF EXISTS trg_check_exhibition_dates_insert;
DROP TRIGGER IF EXISTS trg_check_exhibition_dates_update;
DELIMITER //
CREATE TRIGGER trg_check_exhibition_dates_insert
BEFORE INSERT ON exhibition
FOR EACH ROW
BEGIN
    IF NEW.end_date < NEW.start_date THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'La date de fin doit être postérieure à la date de
début.';
    END IF;
END //

CREATE TRIGGER trg_check_exhibition_dates_update
BEFORE UPDATE ON exhibition
FOR EACH ROW
BEGIN
    IF NEW.end_date < NEW.start_date THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'La date de fin doit être postérieure à la date de
début.';
    END IF;
END //
DELIMITER ;

-- -----
-- Trigger 3 : trg_audit_exhibition_update
-- Objectif : journaliser toute modification d'une exposition

```

```

--          dans une table d'audit (traçabilité).
-- -----
CREATE TABLE IF NOT EXISTS audit_exhibition (
    id          BIGINT          NOT NULL AUTO_INCREMENT,
    exhibition_id BIGINT          NOT NULL,
    old_title    VARCHAR(300),
    new_title    VARCHAR(300),
    old_end_date DATE,
    new_end_date DATE,
    changed_at   DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id)
);

DROP TRIGGER IF EXISTS trg_audit_exhibition_update;
DELIMITER //
CREATE TRIGGER trg_audit_exhibition_update
AFTER UPDATE ON exhibition
FOR EACH ROW
BEGIN
    IF OLD.title    != NEW.title
    OR OLD.end_date != NEW.end_date THEN
        INSERT INTO audit_exhibition
            (exhibition_id, old_title, new_title, old_end_date, new_end_date)
        VALUES
            (OLD.id, OLD.title, NEW.title, OLD.end_date, NEW.end_date);
    END IF;
END //
DELIMITER ;

-- =====
-- SECTION 4 – FONCTIONS ET PROCÉDURES STOCKÉES
-- =====

-- -----
-- Fonction 1 : fn_workshop_participants
-- Objectif : retourne le nombre de participants confirmés
--            à un atelier donné.
-- -----
DROP FUNCTION IF EXISTS fn_workshop_participants;
DELIMITER //
CREATE FUNCTION fn_workshop_participants(p_workshop_id BIGINT)
RETURNS INT
READS SQL DATA
DETERMINISTIC
BEGIN
    DECLARE nb INT DEFAULT 0;
    SELECT COUNT(*) INTO nb
    FROM booking
    WHERE workshop_id = p_workshop_id
    AND payment_status = 'PAID';
    RETURN nb;
END //
DELIMITER ;

-- -----
-- Fonction 2 : fn_avg_rating_artwork
-- Objectif : retourne la note moyenne d'une œuvre (1 décimale).
--            Retourne NULL si aucun avis.
-- -----
DROP FUNCTION IF EXISTS fn_avg_rating_artwork;
DELIMITER //
CREATE FUNCTION fn_avg_rating_artwork(p_artwork_id BIGINT)
RETURNS DECIMAL(3,1)
READS SQL DATA
DETERMINISTIC
BEGIN
    DECLARE avg_r DECIMAL(3,1);

```

```

        SELECT ROUND(AVG(rating), 1) INTO avg_r
        FROM review
        WHERE artwork_id = p_artwork_id;
        RETURN avg_r;
END //
DELIMITER ;

-- -----
-- Procédure 1 : sp_create_workshop
-- Objectif : créer un atelier en s'assurant que l'artiste
--            instructeur existe et est actif.
--            Retourne l'id du nouvel atelier.
-- -----
DROP PROCEDURE IF EXISTS sp_create_workshop;
DELIMITER //
CREATE PROCEDURE sp_create_workshop(
    IN p_title          VARCHAR(300),
    IN p_date_time      DATETIME,
    IN p_duration        INT,
    IN p_max_participants INT,
    IN p_price           DECIMAL(10,2),
    IN p_instructor_name VARCHAR(200),
    IN p_location        VARCHAR(300),
    IN p_level           VARCHAR(50),
    OUT p_new_id         BIGINT
)
BEGIN
    DECLARE v_instructor_id BIGINT;

    SELECT id INTO v_instructor_id
    FROM artist
    WHERE name = p_instructor_name AND is_active = TRUE;

    IF v_instructor_id IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Artiste instructeur introuvable ou inactif.';
    END IF;

    INSERT INTO workshop (title, date_time, duration_minutes, max_participants,
                          price, instructor_id, location, level)
    VALUES (p_title, p_date_time, p_duration, p_max_participants,
            p_price, v_instructor_id, p_location, p_level);

    SET p_new_id = LAST_INSERT_ID();
END //
DELIMITER ;

-- -----
-- Procédure 2 : sp_register_member
-- Objectif : inscrire un membre à un atelier avec vérification
--            de l'existence du membre et de la disponibilité.
--            (Le trigger trg_check_booking_capacity intervient
--            en second filet de sécurité.)
-- -----
DROP PROCEDURE IF EXISTS sp_register_member;
DELIMITER //
CREATE PROCEDURE sp_register_member(
    IN p_member_email    VARCHAR(200),
    IN p_workshop_title  VARCHAR(300),
    OUT p_booking_id     BIGINT
)
BEGIN
    DECLARE v_member_id    BIGINT;
    DECLARE v_workshop_id  BIGINT;

    SELECT id INTO v_member_id

```

```

FROM community_member WHERE email = p_member_email;

SELECT id INTO v_workshop_id
FROM workshop WHERE title = p_workshop_title AND date_time > NOW();

IF v_member_id IS NULL THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Membre introuvable.';
END IF;
IF v_workshop_id IS NULL THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Atelier introuvable ou déjà passé.';
END IF;

INSERT INTO booking (workshop_id, member_id, booking_date, payment_status)
VALUES (v_workshop_id, v_member_id, NOW(), 'PENDING');

SET p_booking_id = LAST_INSERT_ID();
END //
DELIMITER ;

-----
-- Procédure 3 : sp_cancel_booking
-- Objectif : annuler une réservation (passe payment_status
--            à CANCELLED). Ne supprime pas la ligne pour
--            conserver l'historique.
-----
DROP PROCEDURE IF EXISTS sp_cancel_booking;
DELIMITER //
CREATE PROCEDURE sp_cancel_booking(
    IN p_booking_id BIGINT,
    OUT p_result VARCHAR(100)
)
BEGIN
    DECLARE v_status VARCHAR(20);

    SELECT payment_status INTO v_status
    FROM booking WHERE id = p_booking_id;

    IF v_status IS NULL THEN
        SET p_result = 'ERREUR : réservation introuvable.';
    ELSEIF v_status = 'CANCELLED' THEN
        SET p_result = 'INFO : réservation déjà annulée.';
    ELSE
        UPDATE booking SET payment_status = 'CANCELLED' WHERE id = p_booking_id;
        SET p_result = 'OK : réservation annulée.';
    END IF;
END //
DELIMITER ;

```

6. Conclusion

Cette étape a permis d'enrichir la base ArtConnect avec des données réalistes et de la doter d'une couche de fonctionnalités avancées :

- 3 vues : portfolio artistes, programme des expositions, disponibilité des ateliers
- 7 index ciblant les jointures et filtres les plus sollicités
- 4 triggers : capacité des ateliers, cohérence des dates, audit des modifications
- 2 fonctions scalaires réutilisables (participants, note moyenne)
- 3 procédures stockées encapsulant les opérations métier complexes
- 6 scénarios transactionnels dont un ROLLBACK déclenché par trigger

La base est désormais prête pour l'étape 4 : intégration avec l'application Java via JDBC.